

Chapter one

- The goal of AI is to make machine intelligent by studying how human thinks, creating program to do tasks that require human expert, etc.
- **AI researchers** (can a machine act intelligently), **philosophers** (can a machine have consciousness) and **cognitive science** (human intelligence Vs. machine intelligence).
- **Artificial intelligence** is a man-made thinking power. It is about creating an intelligent program.
- Intelligence is composed of reasoning, learning, problem solving, perception, and linguistic.
- **Turing test** (acting humanly) is an operational test to know whether machines can think as well as behave intelligently.
- **Thinking humanly** (cognitive model), **thinking rationally** (laws of thought), **acting rationally** (rational agents).
- Foundations of AI **philosophy** (it gives the concept of how some conclusion can be drawn by a machine), **mathematics** (representation of logic, probability theory, etc.), **economics** (how agents attempt to maximize their well-being), **neuroscience**, **psychology**, **computer engineering**, **control theory and linguistics**.
- Example of AI natural language processing, question answering systems, automatic text summarization, machine translation speech recognition, spam fighting, handwriting recognition and vision system, gaming, recommendation system, fraud detection.
- Challenges in AI **accessibility, integrity, privacy, safety and transparency**.

Chapter two

- **An agent** is anything that perceiving its environment through sensors and acting upon that environment through actuator
- **Rational agent** one that does the right thing that makes it **successful**.
- **Omniscience agent** knows the actual outcome of its action in advance.
- As rationality maximizes, expected performance is gotten and as perfection maximizes actual performance is gotten.
- **Lack of autonomy** happens when agents depends on the prior knowledge of its designer rather than its own percepts.
- **Software agent** w/c works in artificial environment.
- **PEAS** stands for performance, environment, actuators and sensors.
- **Fully** (agent accesses the complete state of the environment/detect all aspects) **and partially observable**.
- **Deterministic** (next state of the environment is determined by the current state) **and stochastic**.
- **Episodic and sequential** (current action may affect all future decisions).
- **Static and dynamic** (always changing environment over time).
- **Discrete** (when there is limited number of states, actions and percepts) **and continuous**
- **Single agent and multi-agent** (competitive and cooperative).
- **Known** (the outcomes for all actions are given) **and unknown environments**.
- **Architecture** is a computing device. **Agent program** is implementation of an agent function.

Chapter three

- **Reflex agent** is simple b/c their actions are based on a direct mapping from states to actions.
- **Model-based reflex agent** works in a partially observable environment and has model that is knowledge about how things happen in the world.
- **Utility based agent** acts based not only goals but also the best way to achieve the goal.
- **Learning agent** agents w/c can learn from their past experience. **Learning element** (makes improvement), **critic, performance element** (selects external action) and **problem generator** (suggests actions)
- **Goal based agent** solves problem by finding sequences of actions that lead to desirable states/goal.
- **Goal formation** is the first step in solving problems.
- Between goal formulation and problem formulation (deciding what actions and states to consider) **states** and **actions** are defined.
- The process of looking for the best sequence/path (**solution**) in the state space is called **search**.
- A problem is defined by **initial state, actions, transition model** (successor function), **goal test**, and **path cost**.
- Initial state, actions and transition model define the **state space**.
- Evaluation criteria for well-defined problem and solution **formulation of a problem as search task, basic search strategies, properties of search strategies, and selection of search strategies**.
- **Fringe** set of search nodes that have not been expanded yet.
- **State space** has unique states while a **search tree** may have cyclic paths.
- A node has **state** (w/c state it's in the state space), **parent-node, action** (which action applied to its parent node to generate it), **path cost** (the cost from initial state to the node n itself), **depth** (number of steps along the path from the initial state).
- Measuring problem solving performance **completeness** (is the strategy guaranteed to find a solution), **optimality, time and space complexity**.
- In AI complexity is expressed in **b** (maximum number of successors of any node/ branching factor), **d** (the depth of the shallowest goal node/ depth of the least-cost solution) and **m** (the maximum length of any path in the state space).

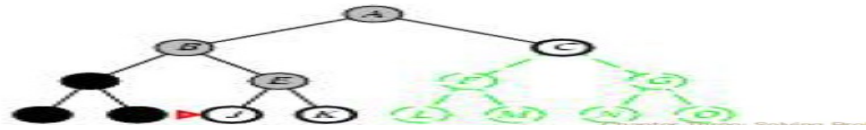
Uninformed search

- In **uninformed search** there is no info about the number of steps or the path cost from the current state to the goal while in **informed/heuristic search** there is a clear info about the path cost. Number of steps from the current state so far.



- **Breadth-first search**
- Breadth-first search is complete, time $O(b^{d+1})$, space $O(b^{d+1})$ and optimal if cost = 1 per step. **Space** is the bigger problem than time. It's based of FIFO data structure.
- **Bread-first** finds the shallowest goal state w/c may not be the least-cost solution and work only if all step costs are equal while, **uniform cost-search** modifies breadth-first strategy by expanding the lowest-cost node.

- **Uniform cost search** complete, if step cost $\geq \epsilon$, time and space $O(b^{\lceil \text{ceiling}(C^*/\epsilon) \rceil})$ and optimal.

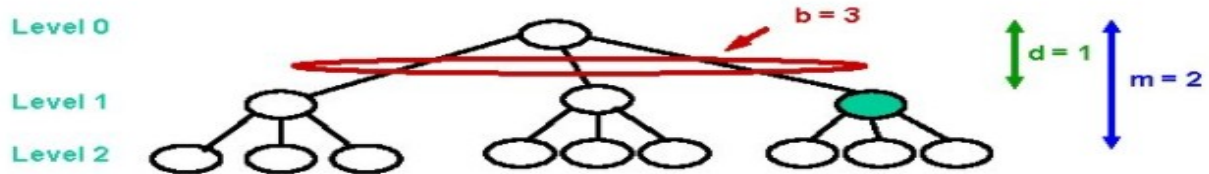


- **Depth-first search**
- Depth first search is not complete if a path is infinite/looping, not optimal, it overcomes time $O(b^m)$ and space $O(bm)$ complexities.
- **Depth limited search** similar to depth first search with a predetermine limit. Three possible outcomes solution, no solution/failure and no solution within cutoff. It's complete but not optimal.
- **Iterative deepening search** is a combination of DFS (**memory efficiency**) and BFS (**fast search**) algorithm. It finds out the best depth limit. It's useful when the **search space is large** and **depth of goal is unknown**. Optimal, complete, time $O(b^d)$ and space $O(bd)$ complexities are reasonable.
- **Iterative lengthening search** uses path cost as limit i.e. has the advantages of uniform cost search.
- **Bidirectional search** one forward from the initial state another backward from the goal then stops when the two searches meet. Time and space complexity = $O(b^{d/2})$. Can be done using BFS, DFS or DLS.
- Avoiding repeated states by not returning to the state it just came from, creating paths with cycles and not generating any generated state.

Informed search (heuristic)

- **Best first search (greedy)** it is good ideally but poor practically since, we cannot make sure a heuristic is good. Not optimal, incomplete, time and space = $O(b^m)$.
- **A* search** evaluates nodes by combining path cost $g(n) \rightarrow$ cheapest known path and heuristic $h(n) \rightarrow$ cheapest estimated path. Is complete, optimal and if $h(n)$ is consistent the value of $f(n)$ along any path are not decreasing.
- **Iterative deepening A* search**
- **Recursive best first search** optimal, space complexity = $O(bd)$.
- **IDA* and RBFS** suffer from using too little memory.
- **Games** are unpredictable so that specifying a move for every possible opponent's reply is a must.
- **Minimax** choose a move to q position with the highest minimax value. It's complete, optimal, time $O(b^m)$ and space $O(bm)$ complexity.
- **Pruning $\alpha - \beta$** does not affect final result (it is exact). With perfect ordering time complexity = $O(b^{m/2})$.
- **Local search algorithm (reading assignment)**

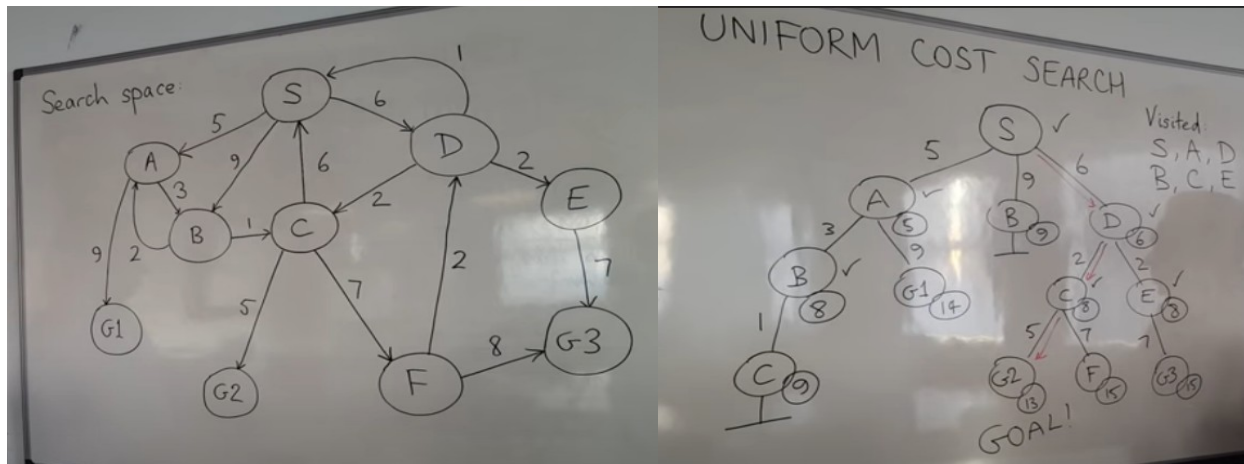
Characterizing Search Algorithms



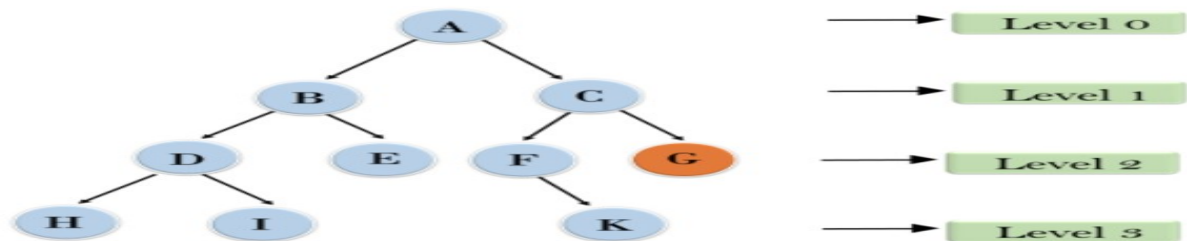
b = maximum branching factor, number of children

d = depth of the shallowest goal node

m = maximum length of any path in the state space



Iterative deepening depth first search



1'st Iteration-----> A

2'nd Iteration-----> A, B, C

3'rd Iteration-----> A, B, D, E, C, F, G

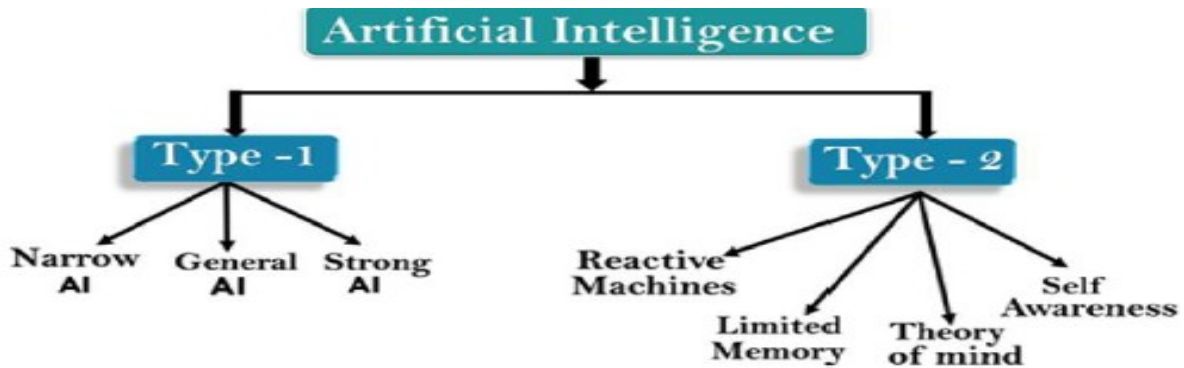
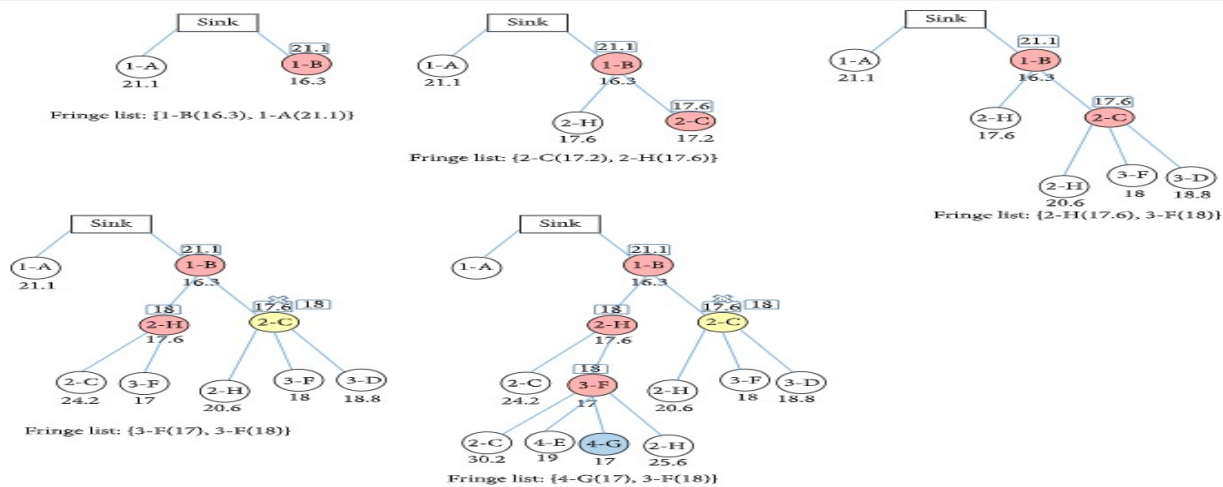
4'th Iteration-----> A, B, D, H, I, E, C, F, K, G

In the fourth iteration, the algorithm will find the goal node.

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes ^a	Yes ^{a,b}	No	No	Yes ^a	Yes ^{a,d}
Time	$O(b^{d+1})$	$O(b^{\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^l)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^{d+1})$	$O(b^{\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(bl)$	$O(bd)$	$O(b^{d/2})$
Optimal?	Yes ^c	Yes	No	No	Yes ^c	Yes ^{c,d}

Figure 3.17 Evaluation of search strategies. b is the branching factor; d is the depth of the shallowest solution; m is the maximum depth of the search tree; l is the depth limit. Superscript caveats are as follows: ^a complete if b is finite; ^b complete if step costs $\geq \epsilon$ for positive ϵ ; ^c optimal if step costs are all identical; ^d if both directions use breadth-first search.

AR-RBFS algorithm steps.



Task Environment	Observable	Deterministic	Episodic	Static	Discrete	Agents
Crossword puzzle	Fully	Deterministic	Sequential	Static	Discrete	Single
Chess with a clock	Fully	Strategic	Sequential	Semi	Discrete	Multi
Poker	Partially	Strategic	Sequential	Static	Discrete	Multi
Backgammon	Fully	Stochastic	Sequential	Static	Discrete	Multi
Taxi driving	Partially	Stochastic	Sequential	Dynamic	Continuous	Multi
Medical diagnosis	Partially	Stochastic	Sequential	Dynamic	Continuous	Single
Image-analysis	Fully	Deterministic	Episodic	Semi	Continuous	Single
Part-picking robot	Partially	Stochastic	Episodic	Dynamic	Continuous	Single
Refinery controller	Partially	Stochastic	Sequential	Dynamic	Continuous	Single
Interactive English tutor	Partially	Stochastic	Sequential	Dynamic	Discrete	Multi